

Authentication, Login, and Role-Based Access

This self-study reading explains how a web application can identify a user, validate login information using MongoDB, remember the logged-in user with a session, and control access based on a user role.

1. Authentication and Authorization

Authentication answers the question: **Who are you?** A login form normally asks for credentials such as an email address and password. The server checks those credentials against account information stored in a database.

Authorization answers the question: **What are you allowed to access?** After a user has been authenticated, the application can check the user's role before allowing access to protected pages or operations.

Authentication = Who are you?

Authorization = What are you allowed to access?

Example roles:

user
admin

2. Full Login Architecture

The browser should not directly check MongoDB. The browser sends login information to the backend API. The backend communicates with MongoDB, validates the credentials, creates a session, and then controls access to protected routes.

```
Login Form
|
v
JavaScript fetch()
|
v
POST /api/login
|
v
Flask
|
v
PyMongo
|
v
MongoDB users collection
|
v
Password validation
|
v
Session
|
v
Role check
|
+---- user  -> User Dashboard
|
+---- admin -> Admin Dashboard
```

3. User Records in MongoDB

Each account can be stored as a MongoDB document. A user document normally contains identity information, a password hash, a role, and an account status.

```
{
  "name": "Student User",
  "email": "user@coffee.local",
  "password_hash": "...",
  "role": "user",
```

```
    "active": true
  }
```

The **role** field is what allows the application to distinguish a normal user from an administrator. The **active** field can be used to disable an account without deleting it.

4. Passwords Should Not Be Stored as Plain Text

A database should not store the original password as readable text. Instead, the password is processed with a password hashing function and the resulting hash is stored.

```
Password entered during account creation
      |
      v
generate_password_hash()
      |
      v
password_hash stored in MongoDB
```

During login, the application checks the submitted password against the stored hash.

```
check_password_hash(
    user["password_hash"],
    submitted_password
)
```

The application does not need to retrieve or display the original password.

5. Sending Login Information From the Webpage

JavaScript reads the values entered in the login form and creates an object.

```
const credentials = {
  email: email,
  password: password
};
```

The object can then be sent to the login API as JSON using a POST request.

```
fetch("/api/login", {
  method: "POST",
  headers: {
    "Content-Type": "application/json"
  },
  body: JSON.stringify(credentials)
});
```

POST is appropriate because the browser is sending login information to the server for processing.

6. Flask Receives the Login Request

The Flask backend reads the JSON sent by the browser.

```
data = request.get_json()

email = data.get("email")
password = data.get("password")
```

The backend then searches MongoDB for an account with the submitted email address.

```
user = users_collection.find_one({
  "email": email
})
```

If no matching account exists, authentication should fail. If the account exists, the backend should also check whether the account is active and whether the submitted password matches the stored password hash.

7. Creating a Session

After a successful login, the application needs to remember that the browser has already authenticated. Flask sessions provide a way to keep information about the logged-in user between requests.

```
session["user_id"] = str(user["_id"])
session["name"] = user["name"]
session["email"] = user["email"]
session["role"] = user["role"]
```

The session can then be checked whenever the browser requests another protected page. Students should think of the session as the application's temporary memory of the logged-in user.

8. Redirecting Based on Role

After the login API returns the user's role, JavaScript can decide which page the browser should open.

```
if (data.role === "admin") {
  window.location.href = "/admin";
} else {
  window.location.href = "/user";
}
```

This redirect improves navigation, but it is not enough to secure the application. A user could manually type the admin URL into the browser. The backend must perform its own role check.

9. Protecting Routes on the Backend

A protected route should first check whether a valid login session exists.

```
if "user_id" not in session:
    return redirect("/")
```

An administrator route should also verify the role.

```
if session.get("role") != "admin":
    return "Forbidden", 403
```

A normal user may be authenticated successfully but still be unauthorized to open an administrator page.

10. Authentication and Authorization Example

Normal User

```
email = user@coffee.local
role = user
```

```
Login succeeds.
User Dashboard is allowed.
Admin Dashboard is not allowed.
```

Administrator

```
email = admin@coffee.local
role = admin
```

```
Login succeeds.
User Dashboard may be allowed.
Admin Dashboard is allowed.
```

11. The Current User Endpoint

A frontend dashboard may need to know who is currently logged in. An endpoint such as GET /api/me can return information stored in the session.

GET /api/me

Possible response:

```
{
  "name": "Student User",
  "email": "user@coffee.local",
  "role": "user"
}
```

If there is no valid session, the API can return HTTP status 401.

12. Logout

Logging out removes the login information stored in the session.

```
session.clear()
```

After the session is cleared, protected pages should require authentication again.

13. Important HTTP Status Codes

```
200 OK
    Request completed successfully.

400 Bad Request
    Required or valid input was not provided.

401 Unauthorized / Not Authenticated
    The request requires a valid login.

403 Forbidden
    The user is logged in but is not allowed
    to access the requested resource.

404 Not Found
    The requested route or resource does not exist.
```

14. Complete Authentication Flow

1. User enters email and password.
2. JavaScript sends:
POST /api/login
3. Flask reads the JSON.
4. PyMongo searches MongoDB:
find_one({"email": email})
5. Flask checks:
 - account exists
 - account is active
 - password hash matches
6. Flask creates the session.
7. API returns the user's role.
8. JavaScript redirects:
user -> /user
admin -> /admin
9. Protected routes check the session.
10. Admin routes also check:
role == "admin"

15. Security Principles for This Activity

The following principles are important even in a classroom application:

- Do not store plaintext passwords.
- Do not put database credentials in frontend JavaScript.
- Do not trust only a frontend redirect for security.
- Protect sensitive routes on the backend.
- Use the session to determine whether a user is logged in.

Check the role on the backend before allowing administrator access.

Return the same general login error for an unknown email and an incorrect password when possible.

16. How to Test the Application

Testing should include successful and unsuccessful cases.

Test 1
Correct normal-user credentials
Expected: User Dashboard

Test 2
Correct admin credentials
Expected: Admin Dashboard

Test 3
Incorrect password
Expected: Login rejected

Test 4
Normal user manually opens /admin
Expected: 403 Forbidden

Test 5
Logout, then open a protected page
Expected: Login required

17. Key Terms

Authentication
Verifying who the user is.

Authorization
Determining what the user is allowed to access.

Credentials
Information used to authenticate, such as email and password.

Password Hash
A protected representation of a password stored instead of the plaintext password.

Session
Information that allows the server to remember the authenticated user between requests.

Role
A value such as "user" or "admin" that is used to determine permissions.

Protected Route
A URL or API endpoint that requires authentication or a specific role.